Casa Ramen

Memoria Descritiva, Explicacao do Codigo e Perguntas para a Defesa

Projeto Final - Programacao Front-End - Proposta B: Restaurante

Ricardo Faria
Ultima atualizacao: 20/09/2026

1. Passos seguidos e design considerado

Escolha da proposta

Escolhi a Proposta B - Restaurante. O cenario pede um site moderno e elegante para um restaurante japones ficticio (Casa Ramen), focado em mostrar a qualidade do menu e das especialidades atraves de imagens e uma navegacao simples e clara.

Ordem de desenvolvimento

- **1.** Estrutura base do projeto (Vite + React + React Router), com as 5 paginas obrigatorias criadas primeiro: Restaurante, Menu, Especialidades, Grupos e Reservas.
- **2.** Extras da proposta, pela ordem em que foram acrescentados: formulario de reservas com envio real de email (EmailJS); paginas legais (Privacidade, Cookies, Aviso Legal) com conteudo real; galeria de imagens na Home e pop-up nas imagens dos pratos; banner de cookies; ligacoes as redes sociais no footer.
- **3.** Fase de testes e correcao de bugs: apos a primeira versao funcional, o site foi revisto de forma sistematica (build, lint e navegacao em browser) para encontrar problemas antes da entrega. Foram corrigidos: falta de tipo de letra base (o site usava o serif por omissao do browser em vez de um tipo moderno), o label "Data"/"Hora" do formulario de reservas que nao ficava alinhado como os restantes campos, a ausencia de uma pagina para enderecos invalidos (mostrava uma pagina em branco), e a tecla Escape que nao fechava o pop-up de imagem.
- **4.** Substituicao das imagens placeholder por fotografias reais dos pratos, com licenca Creative Commons (ver seccao 8), e adicao de um mapa incorporado na pagina de Reservas.
- **5.** Ajustes finais: idioma e titulo corretos no HTML, revisao dos comentarios no codigo, testes de responsividade em varios tamanhos de ecra.

Design considerado

A proposta pede um site que transmita "modernidade e classe". As principais decisoes de design foram: uma paleta de cores quente e sobria (vermelho terracota #7a2518 e castanho escuro #1f1713, tipicos de restaurantes de assinatura), tipografia sans-serif e espacamento consistentes em todas as paginas, e layouts diferentes por pagina mas com a mesma linguagem visual - cumprindo o pedido da proposta de "layouts diferentes, mantendo a coesao". As fotografias reais dos pratos (ver seccao 8) reforcam essa imagem de qualidade e cuidado, em vez das ilustracoes simples usadas na primeira versao do site.

Testes realizados

O site foi testado de forma sistematica, nao so visualmente:

- Build e lint: npm run build e npm run lint executados apos cada alteracao relevante, sem erros nem avisos.
- Navegacao completa: todas as 8 paginas (incluindo as legais) percorridas no browser, incluindo o comportamento de um endereco invalido (agora mostra uma pagina "Pagina nao encontrada" em vez de ficar em branco).
- Responsividade: testada em tres larguras de ecra - telemovel (375px), tablet (768px) e computador - confirmando que os menus, cartoes e galeria se adaptam corretamente em cada uma.
- Formulario de reservas: testado de ponta a ponta, incluindo validacao nativa (ex: email invalido e rejeitado antes de enviar) e confirmacao de que o pedido chega mesmo a caixa de correio real atraves do EmailJS.
- Acessibilidade por teclado: pop-up de imagem testado com a tecla Escape (fecha o pop-up) e navegacao por Tab nos elementos interativos.
- Persistencia: o banner de cookies foi testado a aceitar e a navegar entre paginas, confirmando que a escolha fica guardada e o banner nao volta a aparecer.

2. Arquitetura geral

O projeto e uma Single Page Application (SPA) construida com React e Vite. O browser carrega uma unica pagina HTML (index.html) e e o JavaScript (React) que troca o conteudo consoante o URL, sem recarregar o site - e essa a funcao do React Router.

```
main.jsx  ->  App.jsx  ->  paginas (src/pages)  ->  componentes (src/components)
```

3. Ponto de entrada

main.jsx

E o primeiro codigo a correr. Vai buscar o elemento do index.html, manda o React desenhar <App /> la dentro, e envolve tudo em <BrowserRouter> - e este componente do react-router-dom que da ao App a capacidade de "ler" o URL do browser e decidir que pagina mostrar.

App.jsx

A estrutura-mae do site:

```
<Navbar />                              <- aparece sempre, em todas as paginas
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/menu" element={<Menu />} />
  ...
  <Route path="*" element={<NotFound />} />   <- apanha enderecos invalidos
</Routes>                                <- so isto muda quando navegas
<Footer />                               <- aparece sempre
<CookieBanner />                         <- decide sozinho se se mostra ou nao
```

Navbar e Footer ficam fora de <Routes> de proposito - e assim que garantem que aparecem em todas as paginas, cumprindo o requisito da proposta. A rota final, com path="*", funciona como "apanha-tudo": qualquer endereco que nao corresponda a nenhuma rota especifica cai nela, mostrando a pagina NotFound em vez de uma pagina em branco.

4. Componentes reutilizaveis (src/components/)

- **Navbar.jsx** 5 <Link> para as paginas principais. Link troca de pagina sem recarregar o browser (ao contrario de <a>).
- **Footer.jsx** Copyright + links para as 3 paginas legais (/privacy, /cookies, /legal) + links das redes sociais (abrem em nova aba com target="_blank" rel="noreferrer", por seguranca).
- **Hero.jsx** A faixa grande no topo da Home, com o nome do restaurante.
- **DishCard.jsx** Cartao generico de prato (imagem + nome + preco + descricao). Usado no Menu e nas Especialidades - o mesmo componente, mas o CSS aplica-lhe layouts diferentes consoante a pagina. Recebe titleAs="h2" ou "h3" como prop, para manter a hierarquia de titulos correta.
- **ImagePopup.jsx** Miniatura clicavel que abre a imagem ampliada num overlay (pop-up). Usa useState para controlar aberto/fechado, e um useEffect que ouve a tecla Escape para fechar o pop-up tambem pelo teclado, alem do clique. Usado dentro do DishCard e na galeria da Home.
- **GroupMenu.jsx** Cada menu de grupo na pagina Grupos. Usa a tag nativa HTML <details>/<summary> - o dropdown funciona sem nenhum JavaScript, e o proprio browser que trata do abrir/fechar.

- **ReservationForm.jsx** O formulario de reservas (ver seccao 5).
- **CookieBanner.jsx** Aviso de cookies fixo no fundo do ecra. Usa localStorage (memoria do browser que sobrevive a fechar a pagina) para lembrar que ja aceitaste, e assim nao voltar a aparecer.

5. O formulario de reservas em detalhe

E a peca mais "React" do projeto - usa useState, useRef e uma chamada assincrona ao servico EmailJS:

```
const formRef = useRef(null);                    // referencia direta ao <form>
const [status, setStatus] = useState("idle");     // idle | sending | success | error

function handleSubmit(event) {
  event.preventDefault();                         // impede o recarregar da pagina
  setStatus("sending");
  emailjs.sendForm(SERVICE_ID, TEMPLATE_ID, formRef.current, { publicKey: PUBLIC_KEY })
    .then(() => { setStatus("success"); formRef.current.reset(); })
    .catch(() => setStatus("error"));
}
```

- O EmailJS le os campos diretamente do <form> pelo atributo name de cada input (por isso nao ha um useState por campo).
- As 3 chaves (SERVICE_ID, TEMPLATE_ID, PUBLIC_KEY) vem de variaveis de ambiente (import.meta.env.VITE_...), lidas do ficheiro .env - nunca ficam escritas diretamente no codigo.
- O botao mostra "A enviar..." e fica disabled enquanto status === "sending", para evitar duplo clique.
- Consoante o resultado, aparece uma mensagem verde (sucesso) ou vermelha (erro).
- A validacao nativa do browser (required, type="email", type="date", type="time") impede o envio de dados incompletos ou mal formatados antes mesmo de chamar o EmailJS.
- A conta EmailJS ja esta configurada e testada de ponta a ponta - o email chega mesmo a caixa de correio real da Casa Ramen.

6. Paginas (src/pages/)

- **Home.jsx (/)** Hero + texto de introducao + 3 highlight cards + galeria de 6 fotografias (ImagePopup num grid).
- **Menu.jsx (/menu)** Array menuCategories (Entradas/Ramen/Sobremesas) percorrido com .map() para gerar os DishCard automaticamente, mais a seccao de takeaway.
- **Specialties.jsx (/specialties)** Mesma logica, array specialties, mas usa titleAs="h2" e o CSS da-lhe cartoes maiores e verticais.
- **Groups.jsx (/groups)** Array groupMenus percorrido para gerar GroupMenu, termina com um <Link to="/reservations"> - e o redirecionamento para reservas pedido na proposta.
- **Reservations.jsx (/reservations)** ReservationForm + bloco de informacao (localizacao com mapa do Google Maps incorporado, horario, contactos).
- **NotFound.jsx** Pagina apresentada pela rota "apanha-tudo" (path="*") quando o URL nao corresponde a nenhuma pagina - mostra uma mensagem clara e um link para voltar ao inicio, em vez de uma pagina em branco.
- **PrivacyPolicy / CookiePolicy / LegalNotice.jsx** Paginas estaticas com o conteudo legal, organizadas em <section> com h2 + paragrafo. O Aviso Legal inclui tambem os creditos de imagens (ver seccao 8).

Todas seguem o mesmo padrao: dados num array de objetos no topo do ficheiro, depois um .map() no JSX para gerar o HTML. Isto evita repetir a mesma estrutura varias vezes e torna trivial adicionar ou remover itens.

7. CSS (App.css)

Nao ha CSS Modules nem Styled Components - e um unico ficheiro, organizado por seccoes com comentarios a delimitar "estilos da Home", "estilos do Menu", etc.

- Paleta: vermelho quente #7a2518, fundo escuro #1f1713, creme #fff3df.
- Reset de base: box-sizing: border-box em todos os elementos e um tipo de letra sans-serif explicito no body - sem isto, o browser usava o seu tipo de letra por omissao (um serif tipo Times New Roman), o que nao condizia com o pedido de modernidade da proposta.
- Layouts com flexbox (cards, formulario) e um grid (galeria).
- Reaproveitamento de classes: .dish-card tem o layout base, e .specialties-list .dish-card sobrescreve so o que muda naquela pagina - evita duplicar CSS.
- Media queries em 768px, 600px e 480px para tablet e telemovel.

8. Creditos e licenciamento de imagens

As fotografias dos pratos usadas no Menu, nas Especialidades e na galeria da Home sao fotografias reais, nao ilustracoes, obtidas no Wikimedia Commons sob licenca Creative Commons (CC0, CC BY ou CC BY-SA). Cada foto foi escolhida verificando a sua pagina de licenciamento no proprio Commons antes de ser usada, conforme recomendado na proposta do projeto ("ter em conta o licenciamento das imagens").

- As imagens foram redimensionadas (maximo 1400px no lado maior) e comprimidas para ficheiros JPEG entre 100 e 400 KB, para o site carregar rapidamente sem imagens sobredimensionadas.
- As licencas CC BY e CC BY-SA exigem atribuicao ao autor original. Essa atribuicao esta feita na pagina Aviso Legal do site (seccao "Creditos de imagens"), com o nome do autor, o tipo de licenca e a ligacao a pagina de origem de cada fotografia.
- As fotos com licenca CC0 (dominio publico) - Tonkotsu Ramen, Mochi de Matcha e Tonkotsu Especial - nao exigem atribuicao, mas foram tambem creditadas por transparencia.

Resumo das 10 fotografias usadas:

- **Tonkotsu Ramen -** Syced - CC0 1.0
- **Gyozas de Frango -** Wesoree - CC BY-SA 4.0
- **Edamame -** yomi955 - CC BY 2.0
- **Miso Ramen -** Tail furry - CC BY-SA 4.0
- **Shoyu Ramen -** Guilhem Vellut - CC BY 2.0
- **Mochi de Matcha -** Fumikas Sagisavas - CC0 1.0
- **Dorayaki -** Ocdp - CC BY-SA 3.0
- **Tonkotsu Especial -** Andy Li - CC0 1.0
- **Spicy Miso Ramen -** ~delta - CC BY-SA 4.0
- **Veggie Shoyu Ramen -** Novohoshokoroto (utilizador Wikimedia) - CC BY 4.0

9. Configuracao do projeto

- **vite.config.js** minimo, so ativa o plugin do React.
- **package.json** dependencias: react, react-dom, react-router-dom, @emailjs/browser. Scripts: dev, build, lint, preview.
- **.env / .env.example** chaves do EmailJS, fora do controlo de versoes (.gitignore).
- **.oxlintrc.json** configuracao do linter (oxlint, mais rapido que ESLint).

10. Perguntas provaveis do professor (e respostas)

P: Porque escolheste React em vez de HTML/CSS/JS puro, se a proposta permite qualquer abordagem?
R: React permite dividir a interface em componentes reutilizaveis (ex: DishCard usado em duas paginas diferentes) e o React Router da navegacao entre paginas sem recarregar o site, o que corresponde ao pedido de modernidade da proposta. Continua a usar HTML (JSX), CSS e JavaScript - so organizados de forma diferente.

P: Como funciona a navegacao sem recarregar a pagina?
R: O BrowserRouter (em main.jsx) intercepta os cliques nos componentes <Link> e muda o URL no browser atraves da History API, sem pedir uma nova pagina ao servidor. O App.jsx tem <Routes> que compara o URL atual com cada <Route> e desenha so o componente correspondente.

P: Porque e que a Navbar e o Footer estao fora das <Routes>?
R: Para aparecerem sempre, em todas as paginas, sem ter de os repetir dentro de cada pagina - e um requisito explicito da proposta (mesmo footer/navbar em todas as paginas).

P: O que acontece se o utilizador visitar um endereco que nao existe?
R: A ultima rota definida em App.jsx tem path="*", que funciona como "apanha-tudo": qualquer URL que nao corresponda a nenhuma das rotas anteriores cai nela e mostra a pagina NotFound, com uma mensagem clara e um link para voltar ao inicio. Isto foi adicionado depois de detetar, em testes, que sem essa rota o site mostrava uma pagina em branco.

P: Como funciona o dropdown dos menus de grupo? Porque nao usaste useState?
R: Uso a tag HTML nativa <details>/<summary>. O proprio browser trata da logica de abrir/fechar, sem necessidade de JavaScript, e continua acessivel por teclado. E uma solucao mais simples e robusta do que gerir estado manualmente para algo que o HTML ja resolve.

P: Explica como funciona o formulario de reservas.
R: Uso useRef para ter uma referencia direta ao elemento <form>, e useState para controlar o estado do envio (idle/sending/success/error). No onSubmit, chamo preventDefault() para o browser nao recarregar a pagina, depois emailjs.sendForm() envia os valores dos campos (pelo atributo name de cada input) para o servico EmailJS, que entrega o email. Consoante a resposta, mostro uma mensagem de sucesso ou de erro.

P: Onde ficam as chaves do EmailJS? Porque nao estao no codigo?
R: Ficam num ficheiro .env, lido atraves de import.meta.env.VITE_..., e esse ficheiro esta no .gitignore para nunca ser enviado para o repositorio. Isto evita expor credenciais publicamente.

P: Como garantes que o site e responsivo?
R: O CSS usa flexbox e grid para os layouts, e ha media queries (768px, 600px e 480px) que mudam esses layouts em ecras pequenos - por exemplo, os cards de destaque na Home passam de lado a lado para empilhados, e a galeria de imagens passa de 3 para 2 e depois 1 coluna. Testei manualmente nestas tres larguras antes da entrega.

P: O DishCard e usado em duas paginas com aspetos diferentes - como e possivel sem duplicar codigo?
R: O componente e o mesmo (mesma estrutura JSX), mas o CSS tem seletores mais especificos como .specialties-list .dish-card que sobrescrevem so as propriedades que mudam nessa pagina (tamanho da imagem, direcao do layout), aproveitando a cascata do CSS.

P: Como funciona o banner de cookies e porque nao volta a aparecer depois de aceitar?
R: Uso localStorage, uma memoria do browser que persiste entre visitas. Ao carregar a pagina, um

useEffect verifica se ja existe a chave "cookieConsent"; se nao existir, mostra o banner. Ao clicar em Aceitar, grava essa chave, e da proxima vez o banner nao aparece.

P: Como funciona o pop-up das imagens? Porque tem sempre o mesmo tamanho?
R: O componente ImagePopup guarda um estado open (useState). A miniatura e um <button> que, ao ser clicado, muda open para true e mostra um overlay com a imagem ampliada. Essa imagem ampliada usa sempre a mesma classe CSS (image-popup-enlarged), com largura e altura fixas e object-fit: cover, por isso fica sempre com o mesmo tamanho independentemente da miniatura de origem. Um useEffect adicional ouve a tecla Escape para fechar o pop-up tambem pelo teclado.

P: Onde arranjaste as fotografias dos pratos e podes usa-las legalmente?
R: Sim. Todas as fotografias vem do Wikimedia Commons e tem licenca Creative Commons (CC0, CC BY ou CC BY-SA) - verifiquei a pagina de licenciamento de cada uma antes de a usar. Para as licencas que exigem atribuicao (CC BY e CC BY-SA), inclui os creditos completos (autor, licenca e link para o original) na pagina Aviso Legal do site.

P: Que partes ainda nao estao prontas para producao real?
R: O formulario de reservas ja esta configurado com uma conta EmailJS real e foi testado de ponta a ponta - o email chega mesmo a caixa de correio. As fotografias dos pratos sao agora fotografias reais com licenca Creative Commons, ja nao ilustracoes placeholder. Para um lancamento real, faltaria sobretudo conteudo definitivo do proprio restaurante (morada, horario e fotografias exclusivas da casa, em vez de fotos de terceiros) e a configuracao do servidor de alojamento para redirecionar qualquer URL para o index.html, um requisito tipico de sites de pagina unica (SPA) como este.

P: Como esta organizado o CSS, so um ficheiro?
R: Sim, um unico App.css, organizado em blocos comentados por pagina/componente (Navbar, Home, Menu, Especialidades, etc.), reaproveitando uma paleta de cores comum (vermelho quente #7a2518, fundo escuro #1f1713) para manter coesao visual entre paginas com layouts diferentes, como a proposta pede.